

Banking Management System Using Java OOP Concepts

1. Title

Banking Management System Using Object-Oriented Programming (OOP) in Java

2. Objective

The objective of this project is to implement a simple banking management system using Java Object-Oriented Programming concepts such as:

- Abstraction
- Inheritance
- Polymorphism
- Encapsulation
- Method Overloading
- Method Overriding
- Exception Handling

The system manages different types of bank accounts and performs operations such as displaying account information, calculating interest, depositing money, and withdrawing money with proper error handling.

3. Program Description

This project consists of several classes that work together to simulate a basic banking system.

Classes Used

1. Account (Abstract Class)
2. BankAccount
3. SavingsAccount
4. CurrentAccount
5. InsufficientFundsException

6. Main Class

4. Class Description

4.1 Account (Abstract Class)

The Account class serves as the base class for all account types.

Attributes

Attribute	Type	Access Modifier
accountNumber	int	private
balance	double	private

Methods

- calculateInterest() → Abstract method.
- displayInfo() → Displays account information.
- getAccountNumber() → Returns account number.
- getBalance() → Returns account balance.
- setBalance() → Updates balance.

OOP Concept Used

Abstraction

The class is declared as abstract and contains the abstract method:

```
abstract void calculateInterest();
```

This forces subclasses to provide their own implementation.

4.2 BankAccount Class

The BankAccount class extends the Account class.

Additional Attribute

```
private String holderName;
```

Features

- Stores account holder name.
- Validates initial balance.
- Supports deposit operation.
- Supports withdrawal operation.

- Provides default interest calculation.

Constructor Validation

```
if(balance < 0)
{
    System.out.println("Initial balance cannot be negative.");
    setBalance(0);
}
```

If a negative balance is provided, it is replaced with 0.

4.3 SavingsAccount Class

The SavingsAccount class extends BankAccount.

Interest Calculation

```
double interest = getBalance() * 0.05;
```

Interest rate = **5%**

Overridden Methods

- calculateInterest()
- displayInfo()

This demonstrates **Method Overriding**.

4.4 CurrentAccount Class

The CurrentAccount class extends BankAccount.

Interest Calculation

```
double interest = getBalance() * 0.02;
```

Interest rate = **2%**

Overridden Methods

- calculateInterest()
- displayInfo()

This also demonstrates **Method Overriding**.

4.5 InsufficientFundsException Class

Custom exception class.

```
class InsufficientFundsException extends Exception
```

Purpose:

- Thrown when withdrawal amount exceeds available balance.

Example:

```
throw new InsufficientFundsException(  
    "Insufficient Balance!"  
);
```

4.6 Main Class

The main driver class that executes the program.

Tasks Performed

1. Creates account objects.
 2. Demonstrates polymorphism.
 3. Displays account details.
 4. Performs deposit operations.
 5. Performs withdrawal operation.
 6. Handles exceptions using try-catch-finally.
-

5. OOP Concepts Demonstrated

5.1 Encapsulation

Private variables are protected from direct access.

Example:

```
private int accountNumber;  
private double balance;
```

Access is provided through getters and setters.

5.2 Abstraction

Implemented using:

```
abstract class Account
```

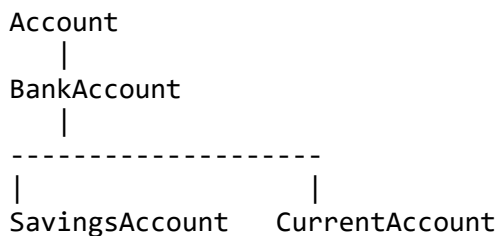
and

```
abstract void calculateInterest();
```

The internal interest calculation details are hidden from users.

5.3 Inheritance

Hierarchy:



SavingsAccount and CurrentAccount inherit features from BankAccount.

5.4 Polymorphism

Implemented through:

```
Account accounts[] = new Account[2];
```

```
accounts[0] = new SavingsAccount(...);  
accounts[1] = new CurrentAccount(...);
```

The same reference type (Account) refers to different objects.

5.5 Method Overloading

Implemented in deposit method:

```
public void deposit(int amount)
```

and

```
public void deposit(double amount)
```

Same method name but different parameter lists.

5.6 Method Overriding

Implemented in:

SavingsAccount
CurrentAccount

Both override:

```
calculateInterest()  
displayInfo()
```

5.7 Exception Handling

Custom exception handling is implemented.

```
try  
{  
    b1.withdraw(7000);  
}  
catch(InsufficientFundsException e)  
{  
    ...  
}  
finally  
{  
    ...  
}
```

The program continues safely even when an error occurs.

6. Program Execution Flow

Step 1

Create Savings Account

```
new SavingsAccount(101,5000,"Saad");
```

Step 2

Create Current Account

```
new CurrentAccount(102,8000,"Rahim");
```

Step 3

Store both accounts in Account array.

Step 4

Display account information using polymorphism.

Step 5

Create another Savings Account.

```
new SavingsAccount(103,1000,"Karim");
```

Step 6

Deposit money.

```
b1.deposit(2000);  
b1.deposit(1000.75);
```

Balance becomes:

$$1000 + 2000 + 1000.75 \\ = 4000.75$$

Step 7

Attempt withdrawal.

```
b1.withdraw(7000);
```

Since:

$$7000 > 4000.75$$

the custom exception is thrown.

Step 8

Exception is caught and handled.

Step 9

Finally block displays current balance.

7. Advantages of the System

- Demonstrates all major OOP concepts.
 - Code reusability through inheritance.
 - Flexible design through abstraction.
 - Secure data handling through encapsulation.
 - Robust error management using custom exceptions.
 - Easy to extend for future account types.
-

8. Conclusion

This project successfully implements a simple Banking Management System using Java. The program demonstrates fundamental Object-Oriented Programming concepts including abstraction, encapsulation, inheritance, polymorphism, method overloading, method overriding, and exception handling. The use of a custom exception improves reliability by preventing invalid withdrawal operations. Overall, the project provides a clear understanding of how OOP principles can be applied to develop a practical banking application.

Account Class :

```
abstract class Account {  
  
    private int accountNumber;  
    private double balance;  
  
    protected Account(int accountNumber, double balance) {  
        this.accountNumber = accountNumber;  
        this.balance = balance;  
    }  
  
    abstract void calculateInterest();  
  
    void displayInfo() {  
  
        System.out.println("Account Number: " + accountNumber);  
        System.out.println("Balance: " + balance);  
  
    }  
  
    protected int getAccountNumber() {  
        return accountNumber;  
    }  
  
    protected double getBalance() {  
        return balance;  
    }  
  
    protected void setBalance(double balance) {  
        this.balance = balance;  
    }  
}
```

BankAccount Class :

```
class BankAccount extends Account {  
  
    private String holderName;  
    public BankAccount(int accountNumber, double balance, String holderName) {  
  
        super(accountNumber, balance);  
  
        if (balance < 0) {  
  
            System.out.println("Initial balance cannot be negative.");  
            setBalance(0);  
        }  
        this.holderName = holderName;  
    }  
  
    public String getHolderName() {  
  
        return holderName;  
    }  
  
    public void setHolderName(String holderName) {  
  
        this.holderName = holderName;  
    }  
    @Override  
    void displayInfo() {  
  
        super.displayInfo();  
  
        System.out.println("Holder Name: " + holderName);  
    }  
    @Override  
    void calculateInterest() {  
  
        double interest = getBalance() * 0.0;  
  
        System.out.println("Interest: " + interest);  
    }  
  
    public void deposit(int amount) {  
  
        if (amount > 0) {  
  
            setBalance(getBalance() + amount);  
        }  
    }  
  
    public void deposit(double amount) {  
  
        if (amount > 0) {  
  
            setBalance(getBalance() + amount);  
        }  
    }  
    public void withdraw(double amount) throws InsufficientFundsException {  
  
        if (amount > getBalance()) {  
            throw new InsufficientFundsException("Insufficient Balance!");  
        }  
  
        setBalance(getBalance() - amount);  
        System.out.println("Withdraw Successful.");  
    }  
}
```

Custom Class:

```
class InsufficientFundsException extends Exception {  
  
    public InsufficientFundsException(String message) {  
        super(message);  
    }  
}
```

SavingsAccount class:

```
class SavingsAccount extends BankAccount {  
  
    public SavingsAccount(int accountNumber, double balance, String holderName) {  
        super(accountNumber, balance, holderName);  
    }  
  
    void calculateInterest() {  
  
        double interest = getBalance() * 0.05;  
        System.out.println("Calculated Interest (5%): " + interest);  
    }  
    void displayInfo() {  
  
        System.out.println("Account Type: Savings Account");  
        super.displayInfo();  
        calculateInterest();  
    }  
}
```

CurrentAccount class:

```
class CurrentAccount extends BankAccount {  
  
    public CurrentAccount(int accountNumber, double balance, String holderName) {  
        super(accountNumber, balance, holderName);  
    }  
  
    void calculateInterest() {  
  
        double interest = getBalance() * 0.02;  
        System.out.println("Calculated Interest (2%): " + interest);  
    }  
  
    void displayInfo() {  
  
        System.out.println("Account Type: Current Account");  
        super.displayInfo();  
        calculateInterest();  
    }  
}
```

Main Class:

```
public class main {
    public static void main(String[] args) {

        Account accounts[] = new Account[2];
        accounts[0] = new SavingsAccount(101, 5000, "Saad");
        accounts[1] = new CurrentAccount(102, 8000, "Rahim");

        for (Account acc : accounts) {
            acc.displayInfo();
            System.out.println(" ");
            System.out.println(" ");
        }

        BankAccount b1 = new SavingsAccount(103, 1000, "Karim");

        b1.deposit(2000);
        b1.deposit(1000.75);

        System.out.println("-----Custom Exception using try-catch Error handling-----");
        System.out.println(" ");

        try {

            b1.withdraw(7000.00);
        }

        catch(ArithmeticException e)
        {
            System.out.println("ArithmeticException : " + e.getMessage());
        }

        catch (InsufficientFundsException e) {

            System.out.println("InsufficientFundsException : " + e.getMessage());
        }
        finally {

            System.out.println("Current balance : " + b1.getBalance());
        }
    }
}
```

Output:

```
Account Type: Savings Account
Account Number: 101
Balance: 5000.0
Holder Name: Saad
Calculated Interest (5%): 250.0

Account Type: Current Account
Account Number: 102
Balance: 8000.0
Holder Name: Rahim
Calculated Interest (2%): 160.0

-----Custom Exception using try-catch Error handling-----

InsufficientFundsException : Insufficient Balance!
Current balance : 4000.75
```